

SBER AI HACK · КОМАНДА «ГУСИ-ЛЕБЕДИ»

# GigaSpec

Самозэволюционирующий SDD-оркестратор на платформе Ouroboros

Держит нить от идеи до PR — и с каждым спринтом эволюционирует ваш процесс сам.

**GigaSpec** — агент, который добывает у команды её собственный процесс разработки, а затем сам ведёт по нему каждую задачу: от сообщения в чате до проверенного pull request. Листайте →

● ПОНЕДЕЛЬНИК, УТРО

● ● ● JIRA · PROJ-1147 · новая задача

Описание: «Сделать фильтр, как обсуждали»

Критерии приёмки: —

Контекст, ссылки: —

Кто и где обсуждал: —

**Какой фильтр?**

в задаче — одна строка

**Где обсуждали?**

автор — на встречах

**Созвон без записи**

каждый понял по-своему

**Через 2 недели — переделка**

«я имел в виду другое»

## Знакомо?

Дело не в Лене и не в постановщике. У команды нет общего ответа на вопрос **«что значит — задача готова к работе?»** — контекст живёт в созвонах и теряется. Итог: две недели работы — и переделка.

● ПАРАДОКС

# Хотим внедрять SDD — а команды не могут описать **свой** процесс.



**Регламент на 40 страниц**

его никто не читает



**Головы сеньоров**

знание живёт там — и теряется



**«Давайте сначала договоримся»**

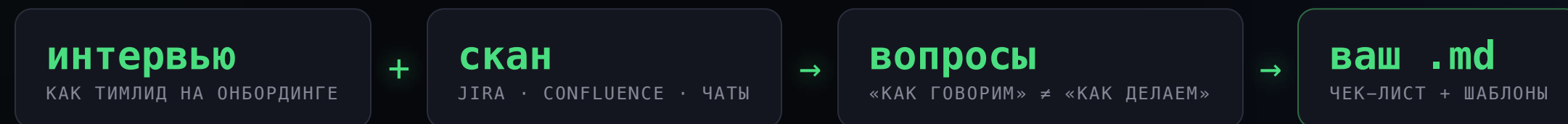
внедрение буксует годами

**Процесс нельзя написать сверху.** Но его можно — добыть.

**SDD (Spec-Driven Development)** — разработка «от задания»: сначала чёткое ТЗ, потом код. Внедрять его хотят ради качества и предсказуемости, но командам он видится бюрократией, а как процесс устроен «именно у нас» — нигде не записано: знание живёт в головах. Значит, его надо оттуда достать.

● ПЕРЕВОРОТ

# Не существует универсального процесса — мы достанем ваш.



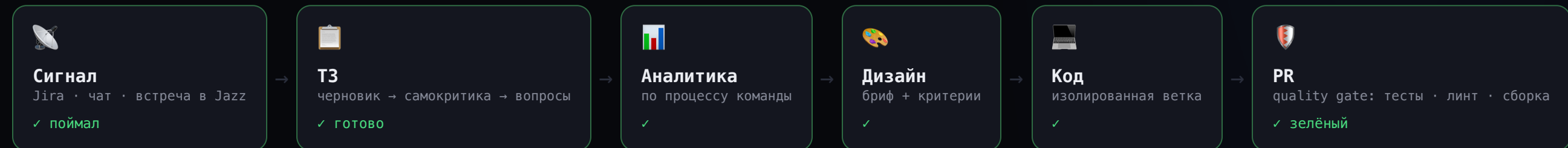
**Первый запуск GigaSpec.** Агент интервьюирует команду, как опытный тимлид на онбординге, и параллельно сканирует Jira, Confluence и чаты. Расхождения между «как говорим» и «как делаем» становятся вопросами, ответы — вашим стандартом: чек-лист и шаблоны в обычных .md-файлах. Правится как заметка — без кода, без консультантов, без сорока страниц.

● КОНВЕЙЕР

# Один движок ведёт задачу от сигнала до PR.

SDD-КОНВЕЙЕР **Готово** ✓ — — —

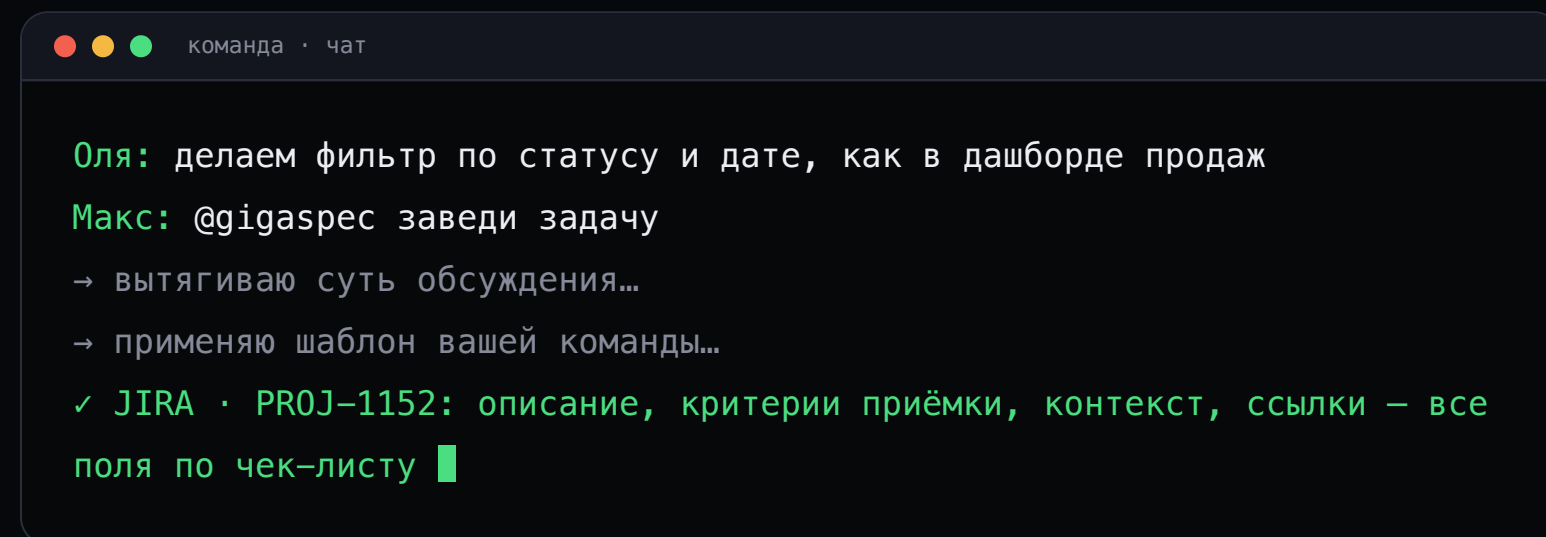
↻ человек на каждом стыке: дальше / поправить / стоп



Пять глаголов: добывает · применяет · охраняет · учит · эволюционирует.

↻ **Каждый этап — чекпоинт с полным контекстом:** можно вернуться, посмотреть, что сделал агент, поправить и перезапустить с этого места. На каждом стыке решает человек: дальше / поправить / стоп. На выходе — проверенный PR, который остаётся только принять.

# Заводит задачи сам — по вашему стандарту.



**Кадр из демо.** Обсуждение в чате не теряется: по одному тегу агент вытягивает суть, применяет шаблон команды — и в Jira появляется полностью оформленная задача. Та же карточка, что была у Лены в начале, — только теперь все поля заполнены.

Просим: «пиши код».  
Он — отказывается.

45% 2

готовность задачи

критичных вопроса не закрыты

СТОП

код не стартует

Он отказывается делать плохо — как хороший инженер.

**Кадр из демо.** Агенту дали размытую задачу и попросили сразу писать код. Он посчитал готовность по чек-листу команды — 45% — и остановился. Закрываем вопросы — и только тогда: ТЗ в Confluence, метка Ready, изолированная ветка, зелёные проверки, PR.





# Самоеволюция

Раз в спринт сам смотрит, что делал и что происходило в задачах, — и предлагает, что улучшить в вашем процессе.

gigaspec · ретро спринта

```
retro: 8 задач вернули из-за отсутствия метрик
→ предложение к checklist.md:
- (пункта нет)
+ метрики успеха определены до старта разработки
тимлид: [ Apply ✓ ] █
```

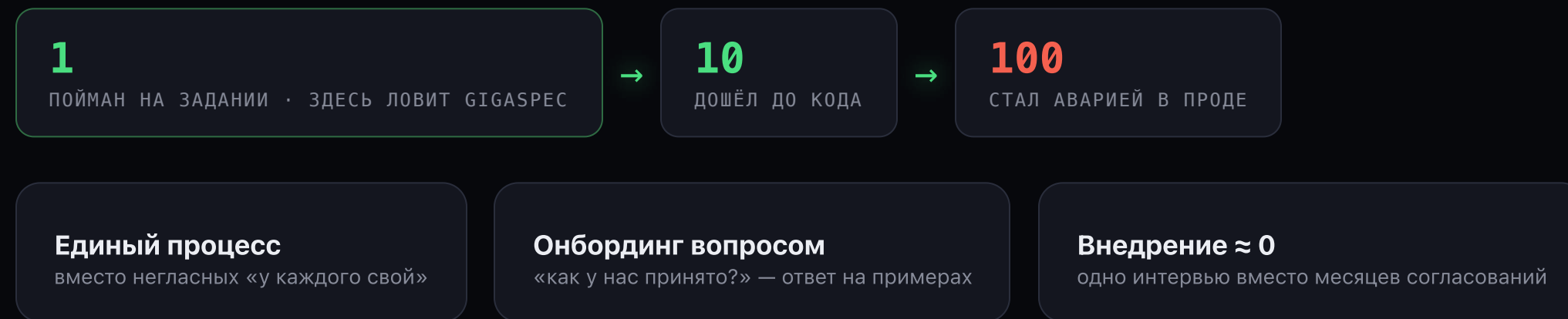
**Кадр из демо — ретро спринта.** Агент проанализировал прожитые задачи, нашёл системную проблему — 8 возвратов из-за отсутствия метрик — и предложил точечную правку чек-листа в виде diff. Процесс живой, но меняется только через человека: тимлид смотрит и жмёт Apply.



● ЭФФЕКТ

Цель — не закрыть задачу побыстрее.  
Цель — сделать предсказуемо и качественно.

ЦЕНА ОДНОГО И ТОГО ЖЕ ДЕФЕКТА ↓



**Правило 1:10:100:** дефект, пойманный на этапе задания, в 10 раз дешевле того же дефекта в коде и в 100 раз дешевле аварии в проде. GigaSpec ловит дефекты на первой ступени — до первой строчки кода.

# Наш код — конфигурация, промпты и **.md**.

**Ouroboros** — агентная платформа, на которой собран GigaСрес: всё показанное ниже — её штатные механизмы, наш вклад — конфигурация, промпты и тонкий plugin-слой. Поэтому тираж прост: новая команда — новое интервью. Ни строчки кода под команду.

## skill **Extension-скилл**

Весь SDD-процесс — в **SKILL.md** и тонком plugin-слое. Правится как текст, не как код.

## cron **scheduled\_tasks**

По расписанию сторожит очередь задач и раз в спринт запускает **ретро**.

## agent **Субагенты-роли**

Каждый этап — своя роль и свой **класс модели** под его вес и цену.

## gate **Quality gate**

PR открывается только на **зелёных** тестах, линте и сборке.

## mcp **MCP Atlassian**

Заводит задачи в **Jira** и пишет ТЗ в **Confluence** напрямую — из коробки.

## chat **telegram-bridge**

Ловит **сигналы** из чатов и шлёт пинги на апрув — канал-агностично.

## git **worktree-изоляция**

Каждая задача пишется в **своей ветке** — не задевая остальные.

## cost **Dashboard · Costs**

Видимая **цена** каждого запуска и жёсткий лимит по бюджету.

● ПРИГЛАШЕНИЕ

Держит нить от идеи до PR.  
И эволюционирует ваш процесс — сам.

```
$ /gigaspec --init
```

`/gigaspec --init` — одна команда, с которой всё начинается: агент приходит в команду и первым делом достаёт её процесс. Дайте нам основной этап — покажем полный цикл вживую.

*Команда «Гуси-Лебеди» · GigaSpec на платформе Ouroboros*